

1. Úloha IOS (2025)

Changelog

- 25.3.2025: (oprava) chybná zmínka o poli `kernel` v specifikaci `list -k` byla nahrazena polem `linux`
- 25.3.2025: (oprava) byly opraveny chybné cesty v příkladech
- 25.3.2025: (zjednodušení implementace) specifikace nyní obsahuje očekávané pořadí argumentů

Předmluva

- Všechny níže uvedené odkazy slouží pouze pro vaši informaci, máte-li zájem dozvědět se více. Obsah za těmito odkazy není pro projekt důležitý.
- Přestože je cílem tohoto projektu získat určité znalosti o moderních zavaděcích Unixu, projekt nevyžaduje restartování systému.

Technické souvislosti

Zavaděč je program zodpovědný za správné načtení operačního systému do paměti při spuštění počítače. Je zodpovědný za správné připojení oddílu obsahujícího binární soubor jádra a (volitelně) počáteční ramdisk (obecně známý jako `initramfs`), spuštění jádra a zároveň předání uživatelem zadaných argumentů jádru. Ačkoli je technicky možné zavést systém přímo bez použití zavaděče, například pomocí [unifikovaných obrazů jádra](#), naprostá většina linuxových distribucí stále používá zavaděč.

Zavaděče, např. GNU GRUB, jsou poměrně složité a umožňují uživateli například interaktivně zvolit operační systém, který se má zavést. Pokud má uživatel nainstalováno více různých operačních systémů, vzniká otázka, jak nakonfigurovat sdílený zavaděč, který načte některý z nainstalovaných systémů. Tato otázka je důsledkem toho, že každý z nainstalovaných operačních systémů potřebuje mít možnost měnit konfiguraci zavaděče, např. když dojde k aktualizaci a systém má nové jádro s jiným názvem. Proto je zapotřebí konfigurační schéma, které je nezávislé na zavaděči (= není vázáno na konkrétní volbu zavaděče). Takové konfigurační schéma je k dispozici ve formátu [Boot Loader Specification](#), který je ve zjednodušené formě základem tohoto projektu.

Specifikace položek zavaděče

Zaváděcí položky systému jsou uloženy v adresáři `/loader/entries` na zaváděcím oddílu.

Poznámka: Přesný význam pojmu „zaváděcí oddíl“ není pro tento projekt důležitý, stejně jako není důležitý, jak takový oddíl na disku najít.

Každá položka zavaděče je uložena v samostatném textovém souboru s příponou `.conf` umístěném přímo v adresáři `/loader/entries` na zaváděcím oddílu (ostatní soubory jsou ignorovány). Soubor se zaváděcími položkami se skládá z několika řádků, z nichž každý má tvar `<key> <value>`. Řetězce `<key>` a `<value>` jsou odděleny alespoň jednou mezerou (znak ASCII s hodnotou `0x20`; jiné druhy whitespace znaků pro oddělení `<key>` a `<value>` nejsou povoleny). Hodnota `<value>` je řetězec začínající prvním znakem jiným než mezera a končící na konci řádku (bez symbolu nového řádku na konci). Klíč `<key>` je jedním z následujících (jiné než níže definované klíče a prázdné řádky ignorujte):

1. `title` - lidsky čitelný název položky zavaděče, který se uživateli zobrazí v nabídce zavaděče.
2. `version` - lidsky čitelná verze položky.
3. `sort-key` - obsahuje řetězec, který se používá pro řazení záznamů. Toto pole je nepovinné.
4. `linux` - obsahuje cestu ke spustitelnému souboru jádra Linuxu.
5. `initrd` - obsahuje cestu k počátečnímu ramdisku (initial ramdisk), který bude použit při zavádění systému.
6. `options` - obsahuje příkazový řádek jádra.
7. `utfit_default` - pokud je hodnota pole `y`, pak je položka výchozí a bude vybrána při dalším spuštění. Toto pole není součástí Boot Loader Specification a je zavedeno pro účely tohoto projektu. Popis toho, jak je výchozí položka uložena v systému GRUB, naleznete v technické poznámce 1. Toto pole není povinné, a pokud v definici položky chybí, položka není výchozí.

Pokud není uvedeno jinak, jsou všechna pole povinná. Je povoleno, aby definice položky obsahovala více řádků se stejným klíčem `<key>`. V takovém případě je hodnota `<value>` odpovídající klíči `<key>` definována posledním z těchto řádků.

Příklad

V systému, který má svůj zaváděcí oddíl připojený na `/boot/`, existuje zaváděcí položka `/boot/loader/entries/4a9c76478b98444fb5e0fbf533950edf-6.12.11-200.fc41.x86_64.conf` s následujícím obsahem (některé řádky nedůležité pro tento projekt jsou odstraněny):

```
title Fedora Linux (6.12.11-200.fc41.x86_64) 41 (Sway)
version 6.12.11-200.fc41.x86_64
linux /vmlinuz-6.12.11-200.fc41.x86_64
initrd /initramfs-6.12.11-200.fc41.x86_64.img
options root=UUID=c5d1643-42a9-4168-a17e-c651f367f367 ro rootflags=subvol=root rd.luks.uuid=luks-57b8f5a1-bd41-45f2-a5c4-12928db060ad rhgb quiet
vutfit_default y
```

Zaváděč systému přečte všechny přítomné položky v daném adresáři, interně sestaví seřazený seznam zaváděcích položek, který se uživateli zobrazí v podobě zaváděcí nabídky, a pak vybraný systém zavede.

Úkol

Vášim úkolem je napsat v shellu nástroj s názvem **bootutil**, který vám umožní vypisovat, upravovat a odstraňovat položky zaváděče systému.

Záznamy jsou uloženy na spouštěcím oddílu, který je v běžícím systému Linux někde připojen (obvykle **/boot**). Nástroj musí podporovat přepínač **-b <boot_entries_dir>**, který uživateli umožní zadat jiný, nekonvenční adresář obsahující definice položek zaváděče. Pokud není přepínač **-b <boot_entries_dir>** použit, nástroj by měl předpokládat, že položky jsou uloženy v adresáři **/boot/loader/entries**. V dalším textu budeme adresář obsahující definice položek zaváděče označovat jako **<boot_entries_dir>**.

*Projekt se snaží být realistický (kromě pole **vutfit_default**), proto ve výchozím stavu skript manipuluje se soubory v adresáři **/boot/Loader/entries**. Pokud tedy vyvíjíte své řešení a zkoušíte ho pod privilegovaným uživatelem, můžete svůj systém znefunkčnit. Pokud spouštíte příkazy jako privilegovaný uživatel, v celém světě UNIXu se implicitně předpokládá, že víte, co děláte. Byli jste upozorněni.*

Výsledný nástroj rozpozná následující příkazy (pozn.: hlavní část hodnocení bude probíhat pomocí skriptů, které budou kontrolovat různé aspekty funkcionality (příkazy, přepínače, ...); za případnou částečnou funkcionalitu tudíž také můžete získat relativně vysoký počet bodů):

1. **list** - Vypíše seznam všech zaváděcích položek zaváděče ve formátu řádků **<title> (<version>, <linux>)**, kde **<title>**, **<version>** a **<linux>** jsou hodnoty polí **title**, **version** a **linux**. Ve výchozím nastavení se položky zobrazují v libovolném pořadí.

Při použití příznaku **-f** se položky zobrazí seřazené abecedně vzestupně podle názvů souborů.

Při použití příznaku **-s** se položky zobrazí abecedně seřazené vzestupně podle hodnoty **sort-key**. Záznamy, které nemají definovaný **sort-key**, se zobrazí za záznamy obsahujícími definici **sort-key** a tyto záznamy jsou seřazeny podle názvu souboru. Pokud existují záznamy, které mají stejnou hodnotu **sort-key**, jsou dále seřazeny podle názvů souborů.

Je-li zadána volba **-k <kernel_regex>**, zobrazí se pouze záznamy, jejichž hodnota pole **linux** odpovídá zadanému rozšířenému regulárnímu výrazu **<kernel_regex>**. Podobně příkaz **-t <title_regex>** zobrazí záznamy, jejichž hodnota pole **title** odpovídá zadanému rozšířenému regulárnímu výrazu **<title_regex>**.

2. **remove <title-regex>** - Odstraní ze systému všechny záznamy, jejichž pole **title** odpovídá zadanému rozšířenému regulárnímu výrazu **<title-regex>**. Lze předpokládat, že každý soubor **.conf** obsahující definici položky je běžným souborem (nikoliv symlinkem/socketem/...).
3. **duplicate [<entry_file_path>]** - Duplikuje položku zaváděče **<entry_file_path>**, pokud je zadána, jinak duplikuje záznam, který je označen jako výchozí (má **vutfit_default** nastaveno na **y**). Pokud neexistuje žádný výchozí záznam a není zadána žádná **<entry_file_path>**, skript se ukončí s nenulovým výstupním kódem. Při modifikaci jsou brány do úvahy následující přepínače (lze jich mít více):

- **-k <kernel_path>** - Zadá novou cestu k jádru (pole **linux**) pro položku.
- **-i <initramfs_path>** - Zadá novou cestu k počátečnímu ramdisku (pole **initrd**) pro položku.
- **-t <new_title>** - Specifikuje nový název (pole **title**) položky.
- **-a <cmdline_args>** - Do výsledné položky přidá argumenty příkazového řádku jádra z **<cmdline_args>**. Popis formátu příkazového řádku jádra naleznete níže.
- **-r <cmdline_args>** - Odstraní argumenty příkazového řádku jádra z **<cmdline_args>** ve výsledném záznamu. Viz níže.
- **-d <destination>** - Cesta k výslednému souboru záznamu.
- **--make-default** - Vytvořený záznam se stane výchozím. Vytvořená položka by měla mít nastavenou hodnotu **vutfit_default** na **y** a v **<boot_entries_dir>** by neměla existovat definice žádné jiné položky, která by byla označena jako výchozí (tj., všechny zbývající položky v daném adresáři mají hodnotu **vutfit_default** nastavenou na **n**).

Pokud není použit přepínač **-d**, tak se výsledný soubor uloží jako nový soubor (s libovolným jménem) do adresáře **<boot_entries_dir>** (nesmí dojít k přepsání jiného souboru v daném adresáři).

V případě, že je některý z přepínačů **-k**, **-i**, **-t** nebo **-d** použit vícekrát, uplatní se pouze poslední výskyt přepínače. Například **./**

`bootutil duplicate -t title1 -t title2` vytvoří položku s názvem `title2`.

Všimněte si, že výsledný výstupní soubor musí být pojmenován jednoznačně, tj. příkaz `duplicate` nemůže omylem přepsat existující soubor se zaváděcí položkou. Jedinou výjimkou je situace, kdy uživatel použije `-d <destination>` s `<destination>` ukazující na již existující soubor – v takovém případě bude soubor `<destination>` přepsán. Doporučujeme vložit do názvu souboru položky zavadeče nějaký infix.

Je možné použít `-d <destination>` s `<destination>`, která nekončí příponou `.conf`. V takovém případě se předpokládá, že uživatel ví, co dělá, a bude vytvořen výstupní soubor `<destination>`.

Poznámka k implementaci: Vaše řešení nemůže k implementaci funkce duplikování používat pevné odkazy (tzv. *hard linky*).

4. `show-default` - Zobrazí položku, která je aktuálně označena jako výchozí (má `vutfit_default` nastaveno na `y`). Pokud taková položka neexistuje, skript se ukončí s nenulovým výstupním kódem. Ve výchozím nastavení se pro každou položku zavadeče zobrazí kompletní soubor definující příslušnou položku bez jakýchkoliv úprav. Příkaz také umožňuje pomocí parametru `-f` zobrazit pouze cestu k souboru definující položku zavadeče označenou jako výchozí (např. pro použití ve skriptech).
5. `make-default <entry_file_path>` - Nastaví položku definovanou v položce `entry_file_path` jako výchozí. Položka, která byla dříve definována jako výchozí, by po provedení příkazu neměla být výchozí. V daném adresáři může existovat pouze jedna výchozí zaváděcí položka.

Formát příkazového řádku jádra

Příkazový řádek jádra je způsob, jakým může správce systému měnit chování jádra a jeho modulů. Například je možné jádru sdělit, aby vytvářelo ladicí zprávy. Příkazový řádek je tvořen řetězcem parametrů jádra oddělených alespoň jednou mezerou (ostatní bílé znaky nejsou povoleny jako oddělovací symboly a jsou považovány za části parametrů). Parametry jádra mohou mít jednu ze tří podob:

- `parameter`
- `parameter=value`
- `module.parameter=value`

Hodnotou parametru může být libovolný řetězec obsahující například `=` nebo mezery. Mezery uvnitř hodnoty (řetězec `value`, tj. řetězec za nejlevějším `=`) je možné chránit uvozovkami nebo escapováním znaků mezery, aby se zabránilo nežádoucímu rozdělení parametrů.

Použití uvozovek nebo escape sekvencí se řídí stejnými pravidly jako jejich použití v shellu. Pokud například chcete použít symbol `"` uvnitř hodnoty, musí být escapován, např. `param=hodnota\"` nebo je možné použít `param='hodnota''`.

Příklady:

- `root=UUID=c5d1643-42a9-4168-a17e-c651f367f367 ro rootflags=subvol=root rhgb quiet`
- `root=UUID=c5d1643-42a9-4168-a17e-c651f367f367 rhgb quiet a="b c" root=UUID=de123456-aaaa-ffff-1111-c651f367f367`
- `root=UUID=c5d1643-42a9-4168-a17e-c651f367f367 rhgb quiet a=b\ c`
- `root=UUID=c5d1643-42a9-4168-a17e-c651f367f367 rhgb quiet a='b c'`

Doporučujeme použít příkaz `xargs`, který správně analyzuje příkazový řádek jádra. Chcete-li například zobrazit parametry jádra tak, aby byl každý parametr jádra vypsán na samostatném řádku, použijte:

```
my_cmdline='root=UUID=c5d1643-42a9-4168-a17e-c651f367f367 rhgb quiet a="b c"'
echo "$my_cmdline" | xargs -n1
```

s výstupem

```
root=UUID=c5d1643-42a9-4168-a17e-c651f367f367
rhgb
quiet
a=b c
```

Všimněte si, že `xargs` odstraňuje uvozovky kolem hodnot (pokud jsou použity), takže v případě potřeby je může být nutné znovu zavést.

Úprava příkazového řádku jádra

- **Přidání parametrů.** Pokud je daný `parameter` (resp. `parameter=value`) již přítomen na příkazovém řádku jádra, neměly by být provedeny žádné změny. Například pokud máme příkazový řádek `root=xyz foo=bar` a uživatel chce přidat `foo=bar`, pak by se příkazový řádek neměl měnit. Naopak přidáním `foo2` do příkazového řádku jádra vznikne nový příkazový řádek s hodnotou `root=xyz foo=bar foo2`. Výsledkem přidání parametru `foo=bar2` do příkazového řádku je `root=xyz foo=bar foo=bar2`.
- **Odstranění parametrů.** Při odstraňování parametru `foo` z příkazového řádku by měly být z příkazového řádku jádra odstraněny

všechny výskyty `foo` a `foo=*` (kde `*` značí libovolnou hodnotu). Máme-li například příkazový řádek `root=xyz foo=bar1 foo=bar2 foo=bar3` a uživatel chce odstranit parametr `foo`, pak výsledný příkazový řádek bude `root=xyz`. Na druhou stranu, pokud chce uživatel odstranit `foo=bar`, pak by měly být odstraněny pouze výskyty `foo=bar`. Odstraněním `foo=bar1` z výše uvedeného příkazového řádku vznikne `root=xyz foo=bar2 foo=bar3`. V případě, že se `parameter` (nebo `parameter=hodnota`) vyskytuje v příkazovém řádku jádra vícekrát, musí být odstraněny všechny výskyty.

Oba přepínače (`-a` a `-r`) přijímají seznam parametrů, které se přidávají ve formátu příkazového řádku jádra. Například `bootutil duplicate -a 'p1=v1 p2=v2'` je platné volání, ekvivalentní volání `bootutil duplicate -a p1=v1 -a p2=v2`. Volání `bootutil duplicate -a 'p1=v1' -a 'p1=v1'` je ekvivalentní `bootutil duplicate -a p1=v1`, tj. parametr `p1=v1` se do příkazového řádku přidá pouze jednou, pokud již není přítomen na příkazovém řádku jádra.

Při použití příkazu `duplicate` je možné použít přepínače `-a` i `-r` vícekrát a v libovolném pořadí (přidání/odebrání parametrů se pak provede v tomto pořadí). Například příkaz `bootutil duplicate -a param=value -r param <entry>` duplikuje danou položku definovanou v `<entry>`, přidá parametr `foo=bar` a poté odstraní všechny výskyty parametru `foo`. Proto parametr `-a foo=bar` nemá žádný účinek. Příkaz `duplicate -r param -a param=value` nejprve odstraní všechny výskyty parametru `param` a poté přidá do příkazového řádku parametr `param=value`. Spuštěním příkazu `duplicate -r foo -a foo=value` se nejprve odstraní všechny výskyty parametru `foo` a poté se do příkazového řádku přidá `foo=value`. Například při spuštění výše uvedeného příkazu pro duplikaci položky zavaděče s příkazovým řádkem jádra `root=xyz foo=bar1 foo=bar2 foo=bar3` se vytvoří položka s hodnotou příkazového řádku `root=xyz foo=value`.

Příklady

V adresáři `/boot/loader/entries/` máme definovány tři položky zavaděče s následujícím obsahem:

```
$ cat /boot/loader/entries/entry\ 1.conf
title Fedora Linux 41
version 6.12.5-200.fc41.x86_64
linux /vmlinuz-6.12.5-200.fc41.x86_64
initrd /initramfs-6.12.5-200.fc41.x86_64.img
options root=UUID=cf5d1643-42a9-4168-a17e-c651f367f367rhgb quiet
vutfit_default y
sort-key bbb
```

```
$ cat /boot/loader/entries/entry\ 2.conf
title Fedora Linux 41 with debug
version 6.12.5-200.fc41.x86_64+debug
linux /vmlinuz-6.12.5-200.fc41.x86_64
initrd /initramfs-6.12.5-200.fc41.x86_64.img
options root=UUID=cf5d1643-42a9-4168-a17e-c651f367f367rhgb quiet debug
vutfit_default n
sort-key aaa
```

```
$ cat /boot/loader/entries/entry\ 3.conf
title Linux from Scratch
version 1.0
linux /vmlinuz-5.11.1.x86_64
initrd /initramfs-5.11.1.x86_64
options root=UUID=cf5d1643-42a9-4168-a17e-c651f367f367rhgb quiet debug
vutfit_default n
```

Následují příklady spuštění skriptu.

bootutil list

```
$ bootutil list -s
Fedora Linux 41 with debug (6.12.5-200.fc41.x86_64+debug, /vmlinuz-6.12.5-200.fc41.x86_64)
Fedora Linux 41 (6.12.5-200.fc41.x86_64, /vmlinuz-6.12.5-200.fc41.x86_64)
Linux from Scratch (1.0, /vmlinuz-5.11.1.x86_64)
```

bootutil duplicate

Spuštění

```
$ bootutil duplicate -a param=value -r debug -t 'Linux from Scratch (fixed)' --make-default /boot/loader/entry\ 3.conf
```

vytvoří novou položku (pojmenovanou například `/boot/loader/entry\ 3.1.conf`) s následujícím obsahem:

```
title Linux from Scratch (fixed)
version 1.0
linux /vmlinuz-5.11.1.x86_64
initrd /initramfs-5.11.1.x86_64
options root=UUID=c5d1643-42a9-4168-a17e-c651f367f367rhgb quiet param=value
vutfit_default y
```

Všechny ostatní položky budou mít `vutfit_default` nastaveno na `n`.

Implementační detaily

1. Pro třídění použijte příkaz `sort`.
2. Všechny regulární výrazy, které může zadat uživatel, jsou rozšířené a lze je použít v `grep -E`.
3. Skript by neměl měnit ani vytvářet žádné soubory (včetně dočasných) mimo `<boot_entries_dir>`. Podobně by dílčí příkazy měly měnit pouze soubory, které je nezbytně nutné upravit. Například příkaz `show-default` by neměl měnit žádné soubory.
4. Skript by měl běžet na všech běžných shellech (`dash`, `ksh`, `bash`). Pokud použijete vlastnost specifickou pro nějaký shell, uveďte to pomocí direktivy interpretu na prvním řádku souboru, např. `#!/bin/bash` nebo `#!/usr/bin/env bash` pro `bash`. Můžete použít GNU rozšíření pro `sed` či `awk`. Jazyky Perl, Python, Ruby, atd. povoleny nejsou.

UPOZORNĚNÍ: některé servery, např. `merlin.fit.vutbr.cz`, mají symlink `/bin/sh -> bash`. Ověřte si proto, že skript skutečně testujete daným shellem. Doporučujeme ověřit správnou funkčnost pomocí virtuálního stroje níže.

5. Referenční systém je k dispozici na virtuálním stroji [zde](#) (user/password: `ios/ios-shell`).
6. je možné očekávat, že Váš skript bude spuštěn následovně: `./bootutil [ -b <boot_entries_dir> ] PŘÍKAZ ...argumenty příkazu...`

FYI - Jak věci fungují ve skutečnosti

Technická poznámka 1

Specifikace zaváděcího modulu nedefinuje koncept výchozí položky, proto si každý zavaděč může vymyslet jiný způsob uložení výchozího záznamu (například jednoduše vzít první dostupný záznam). Zavaděč [GNU GRUB](#) obvykle používá speciální soubor `grubenv` umístěný v `/boot/grub/grubenv` nebo `/boot/efi/EFI/<vaše-distribuce>/grubenv` v případě, že používáte UEFI místo starého BIOS-u. Tento minimalistický soubor obsahuje na rozdíl od konfiguračního souboru `grub.cfg` pouze definice proměnných GRUB-u, jako jsou `default_entry` nebo `saved_entry`, bez jakékoliv logiky navíc. Upozorňujeme, že do `grubenv` nelze psát ručně např. pomocí editoru `vim` a je potřeba s ním manipulovat pomocí příkazu `grub2-editenv`. I když soubor otevřete ve `vim`-u pouze pro čtení a poté jej ukončíte pomocí `:wq`, `vim` automaticky připojí nový řádek ke konci souboru, a tím jej poruší.

Built with [Pandoc](#) using [pandoc-bootstrap](#) theme